EGCO 213

Group Project 2 – Insurance Claims and Payments

The project can be done in a group of <= 5 students. Each group must do the project by themselves:

- **Everyone involved in cheating, either as a source or copier, will get ZERO point.**
- If late submitting group copies code from a graded group, the graded group will still be penalized.
- **In case of AI usage, you must write your own prompts and perform your own acquisition with the AI.** Due to the dynamic nature of generative AI, it is very unlikely that any 2 groups will get identical generated code even when using identical prompts.
  - Therefore, submitting identical code will be counted as cheating.
  - Don't use generated content obtained by other groups as your own.
  - Don't share generated content you get from the AI with other groups.
  - If any suspicious arises, I may ask both groups to show their chat history with the AI. Failure to do so will result in cheating penalty.
- And remember, you must be able to orally explain every line of the code you submit, regardless of its origin.

1. This project uses only 1 input file config.txt. Lines beginning with "#" = comments; don't remove them.

```
# First column in every parameter line = label
#
# Label followed by: days of simulation
days,                       6
#
# ClaimGateways
# Label followed by: number of gateways, batch size per submission
ClaimGateways_num_batch,    3, 50
#
# PaymentGateways
# Label followed by: number of gateways, daily payment tokens
PaymentGateways_num_max,    2, 80
#
# BrokerThreads
# Label followed by: number of threads, min daily claims, max daily claims
BrokerThreads_num_min_max,  3, 20, 100
#
# AssessorThreads
# Label followed by: number of threads, daily capacity
AssessorThreads_num_max,    3, 80
```

Don’t hard code these values. I may change some of them to check whether your calculation is correct.

- There are always 5 parameter lines with columns as stated above, but numbers may be changed.
- There won’t be any input error (e.g. invalid input, negative number, wrong format, missing columns) in this file. But the program must still handle the case of missing file. Don’t let it crash.

2. Implement **class ClaimGateway** representing an individual claim gateway. The number of gateways and batch size for claim submissions (from BrokerThreads) are read from config file. This class should have a variable to keep pending claims. You may keep ClaimGateway objects in an ArrayList.

3. Implement **class PaymentGateway** representing an individual payment gateway. The number of gateways and daily payment tokens for payment orders (by AssessorThreads) are read from config file. This class should have a variable to keep remaining tokens. You may keep PaymentGateway objects in another ArrayList.

4. Implement **class BrokerThread** representing an individual insurance broker as thread. Thread activities are done in loop. Each iteration of a loop = 1 day. In each day, BrokerThreads must wait for each other to perform steps 4.1-4.3 together.

- 4.1 Wait until one thread (can be main, BrokerThread, or AssessorThread) prints day number, pending claims at each ClaimGateway (accumulated from previous days), and payment tokens at each PaymentGateWay (reset every day).
- 4.2 Simulate new claim arrivals to each BrokerThread:
  - 4.2.1 Random new claims between min and max daily claims (from config file).
  - 4.2.2 Split new claims into batches according to batch size allowed by ClaimGateway.
  - 4.2.3 Show output from 4.2.1 and 4.2.2 as in demo.
- 4.3 Simulate claim submissions to ClaimGateway by each BrokerThread:
  - 4.3.1 Random 1 ClaimGateway. Submit new claims in batches to ClaimGateway and update pending claims at the gateway. Batch submissions by each thread don't need to be consecutive.
  - 4.3.2 Show output from 4.3.1 as in demo.

** All BrokerThreads must work on the same list of ClaimGateways.

5. Implement **class AssessorThread** representing an individual claim assessor as thread. Thread activities are done in loop. Each iteration of a loop = 1 day. In each day, AssessorThreads must wait for each other to perform steps 5.1-5.4 together.

- 5.1 Wait until one thread (can be main, BrokerThread, or AssessorThread) finishes activities in 4.1. Also wait until all BrokerThreads finish submitting new claims to ClaimGateways.
- 5.2 Simulate claim approvals at ClaimGateway by each AssessorThread:
  - 5.2.1 Random 1 ClaimGateway. Random claims to approve between 1 and either daily capacity (from config file) or pending claims at the gateway. The random number can be 0 if there is no claim at the gateway; otherwise, there must be at least 1 claim.
  - 5.2.2 Approve claims taken from the gateway and update pending claims at the gateway.
  - 5.2.3 Show output from 5.2.2 as in demo.
  - 5.3.4 For simplicity, each thread will log on to only 1 ClaimGateway in each day – if it can't approve any claim from this gateway, it won't try any other.
- 5.3 Update pending payments = pending payments from previous days + approved claims. Show output from this step as in demo. If pending payments is not 0, proceed to step 5.4; otherwise, complete today.
- 5.4 Simulate payment issues to PaymentGateway by each AssessorThread:
  - 5.4.1 Random 1 PaymentGateway. Thread must acquire 1 token from the gateway to issue 1 payment (i.e. 1 token = 1 successful payment). It will try to acquire as many tokens, to issue as many pending payments, as possible. Remaining pending payments are accumulated to next days.
  - 5.4.2 Update successful payments by each thread (will be used in summary report) and remaining tokens at the gateway.
  - 5.4.3 Show output from 5.4.2 as in demo.
  - 5.4.4 For simplicity, each thread will log on to only 1 PaymentGateway in each day – if it can't get any token from this gateway, it won't try any other.

** All AssessorThreads must work on the same lists of ClaimGateways and PaymentGateways.

6. Implement main class with main method.
   6.1 Read simulation parameters from config.txt.
   6.2 Create ClaimGateways, PaymentGateways, BrokerThreads, AssessorThreads according to the parameters. You are recommended to use ArrayLists to keep objects for flexibility. Start all threads.
   6.3 After all threads complete all days of simulation, let main thread report successful payments by each AssessorThread as in the demo. The report must be sorted in decreasing order of successful payments, and by thread names (if successful payments are equal).

** Everything printed to the screen must be labelled by the name of the thread who prints it. Don't hard code thread's name but use Thread.currentThread().getName().

7. Package and folder structure must be correct.
   7.1 Your source files (.java) must be in folder Project2_XXX where XXX = full ID of the group representative, assuming that this folder is under Maven's "src/main/java" structure. The first lines of all source files must be comments containing names & IDs of all members.
   7.2 Input files must be read from Project2_XXX. Don't use absolute path that is valid only on your PC.
   7.3 Add readme.txt containing names & IDs of all members in Project2_XXX.

Submission

1. Group representative zips and submits Project2_XXX to Google classroom
2. Other members submit only readme.txt to Google classroom

Grading

1	point	new day	(pending claims, tokens)
1.5	points	daily processing by BrokerThreads	(split claims, submit claims + batch submissions)
1.5	points	daily processing by AssessorThreads	(approve claims, check pending payments, issue payments)
2	points	summary by main + others	(successful payments + sorted, missing file, thread name)
4	points	proper design & programming in OOP style Maximum & safe concurrency is expected. Don't enforce sequential execution when threads can work in parallel. For example, if 2 BrokerThreads want to submit claims to different ClaimGateways, they should be able to do it concurrently.	

Late submission: -0.5 points for <1 week late; -1 point for each 1 full week late

```
--- exec:3.1.0:exec (default-cli) @ solutions ---
java.io.FileNotFoundException: src\main\java\Project2\config.txt (The system cannot find the file specifie
New file name =   config1
java.io.FileNotFoundException: src\main\java\Project2\config1 (The system cannot find the file specified)
New file name =   config_1
java.io.FileNotFoundException: src\main\java\Project2\config_1 (The system cannot find the file specified)
New file name =   config_1.txt
            main  >>  ================================== Parameters ==================================
            main  >>  Days of simulation : 6
            main  >>  ClaimGateways      : [ClaimGateway_0, ClaimGateway_1, ClaimGateway_2]
            main  >>    Batch size       : 50
            main  >>  PaymentGateways    : [PaymentGateway_0, PaymentGateway_1]
            main  >>    Daily tokens     : 80
            main  >>  BrokerThreads      : [BrokerThread_0, BrokerThread_1, BrokerThread_2]
            main  >>    Daily claims     : min = 20, max = 100
            main  >>  AssessorThreads    : [AssessorThread_0, AssessorThread_1, AssessorThread_2]
            main  >>    Daily capacity   : 80
            main  >>
            main  >>  ================================================================================
            main  >>  Day 1
            main  >>  ClaimGateway_0 pending claims =     0
            main  >>  ClaimGateway_1 pending claims =     0
            main  >>  ClaimGateway_2 pending claims =     0
            main  >>  reset PaymentGateway_0 tokens =    80
            main  >>  reset PaymentGateway_1 tokens =    80
            main  >>
  BrokerThread_1  >>  new claims =  87     split to [50, 37]
  BrokerThread_2  >>  new claims =  39     split to [39]
  BrokerThread_0  >>  new claims =  97     split to [50, 47]
  BrokerThread_0  >>  submit  50 claims to ClaimGateway_0       ClaimGateway_0 pending claims =    50
  BrokerThread_2  >>  submit  39 claims to ClaimGateway_1       ClaimGateway_1 pending claims =    39
  BrokerThread_0  >>  submit  47 claims to ClaimGateway_0       ClaimGateway_0 pending claims =    97
  BrokerThread_1  >>  submit  50 claims to ClaimGateway_0       ClaimGateway_0 pending claims =   147
  BrokerThread_1  >>  submit  37 claims to ClaimGateway_0       ClaimGateway_0 pending claims =   184
  BrokerThread_1  >>
AssessorThread_2  >>  approve  54 claims from ClaimGateway_0    ClaimGateway_0 pending claims =   130
AssessorThread_0  >>  approve   0 claims from ClaimGateway_2    ClaimGateway_2 pending claims =     0
AssessorThread_1  >>  approve   0 claims from ClaimGateway_2    ClaimGateway_2 pending claims =     0
AssessorThread_1  >>  pending payments =   0
AssessorThread_0  >>  pending payments =   0
AssessorThread_2  >>  pending payments =  54
AssessorThread_2  >>  issue  54 payments to PaymentGateway_0    PaymentGateway_0 tokens =  26
            main  >>
            main  >>  ================================================================================
            main  >>  Day 2
            main  >>  ClaimGateway_0 pending claims =   130
            main  >>  ClaimGateway_1 pending claims =    39
            main  >>  ClaimGateway_2 pending claims =     0
            main  >>  reset PaymentGateway_0 tokens =    80
            main  >>  reset PaymentGateway_1 tokens =    80
            main  >>
  BrokerThread_0  >>  new claims =  61     split to [50, 11]
  BrokerThread_2  >>  new claims =  46     split to [46]
  BrokerThread_1  >>  new claims =  47     split to [47]
  BrokerThread_1  >>  submit  47 claims to ClaimGateway_0       ClaimGateway_0 pending claims =   177
  BrokerThread_2  >>  submit  46 claims to ClaimGateway_1       ClaimGateway_1 pending claims =    85
  BrokerThread_0  >>  submit  50 claims to ClaimGateway_0       ClaimGateway_0 pending claims =   227
  BrokerThread_0  >>  submit  11 claims to ClaimGateway_0       ClaimGateway_0 pending claims =   238
  BrokerThread_0  >>
AssessorThread_2  >>  approve   0 claims from ClaimGateway_2    ClaimGateway_2 pending claims =     0
AssessorThread_0  >>  approve   0 claims from ClaimGateway_2    ClaimGateway_2 pending claims =     0
AssessorThread_1  >>  approve   0 claims from ClaimGateway_2    ClaimGateway_2 pending claims =     0
AssessorThread_1  >>  pending payments =   0
AssessorThread_0  >>  pending payments =   0
AssessorThread_2  >>  pending payments =   0
            main  >>
```

Handle missing file/incorrect file name

Path must still be correct

- Split claims to batches
- Batch submissions don't need to be consecutive, depending on thread competition

No pending payment, no work today

- Pending claims are accumulated from previous days
- Tokens are reset every day

```
            main  >>  ================================================================================
            main  >>  Day 3
            main  >>  ClaimGateway_0 pending claims =   238
            main  >>  ClaimGateway_1 pending claims =    85
            main  >>  ClaimGateway_2 pending claims =     0
            main  >>  reset PaymentGateway_0 tokens =    80
            main  >>  reset PaymentGateway_1 tokens =    80
            main  >>
  BrokerThread_2  >>  new claims =  54     split to [50, 4]
  BrokerThread_1  >>  new claims =  61     split to [50, 11]
  BrokerThread_0  >>  new claims =  53     split to [50, 3]
  BrokerThread_0  >>  submit  50 claims to ClaimGateway_1        ClaimGateway_1 pending claims =   135
  BrokerThread_1  >>  submit  50 claims to ClaimGateway_0        ClaimGateway_0 pending claims =   288
  BrokerThread_1  >>  submit  11 claims to ClaimGateway_0        ClaimGateway_0 pending claims =   299
  BrokerThread_0  >>  submit   3 claims to ClaimGateway_1        ClaimGateway_1 pending claims =   138
  BrokerThread_2  >>  submit  50 claims to ClaimGateway_1        ClaimGateway_1 pending claims =   188
  BrokerThread_2  >>  submit   4 claims to ClaimGateway_1        ClaimGateway_1 pending claims =   192
  BrokerThread_2  >>
AssessorThread_0  >>  approve  14 claims from ClaimGateway_1     ClaimGateway_1 pending claims =   178
AssessorThread_1  >>  approve  15 claims from ClaimGateway_0     ClaimGateway_0 pending claims =   284
AssessorThread_2  >>  approve  65 claims from ClaimGateway_0     ClaimGateway_0 pending claims =   219
AssessorThread_2  >>  pending payments =  65
AssessorThread_1  >>  pending payments =  15
AssessorThread_0  >>  pending payments =  14
AssessorThread_0  >>  issue  14 payments to PaymentGateway_1     PaymentGateway_1 tokens =  66
AssessorThread_1  >>  issue  15 payments to PaymentGateway_0     PaymentGateway_0 tokens =  65
AssessorThread_2  >>  issue  65 payments to PaymentGateway_1     PaymentGateway_1 tokens =   1
            main  >>
            main  >>  ================================================================================
            main  >>  Day 4
            main  >>  ClaimGateway_0 pending claims =   219
            main  >>  ClaimGateway_1 pending claims =   178
            main  >>  ClaimGateway_2 pending claims =     0
            main  >>  reset PaymentGateway_0 tokens =    80
            main  >>  reset PaymentGateway_1 tokens =    80
            main  >>
  BrokerThread_0  >>  new claims =  87     split to [50, 37]
  BrokerThread_1  >>  new claims =  81     split to [50, 31]
  BrokerThread_2  >>  new claims =  53     split to [50, 3]
  BrokerThread_2  >>  submit  50 claims to ClaimGateway_2        ClaimGateway_2 pending claims =    50
  BrokerThread_2  >>  submit   3 claims to ClaimGateway_2        ClaimGateway_2 pending claims =    53
  BrokerThread_0  >>  submit  50 claims to ClaimGateway_2        ClaimGateway_2 pending claims =   103
  BrokerThread_0  >>  submit  37 claims to ClaimGateway_2        ClaimGateway_2 pending claims =   140
  BrokerThread_1  >>  submit  50 claims to ClaimGateway_2        ClaimGateway_2 pending claims =   190
  BrokerThread_1  >>  submit  31 claims to ClaimGateway_2        ClaimGateway_2 pending claims =   221
  BrokerThread_1  >>
AssessorThread_2  >>  approve  36 claims from ClaimGateway_1     ClaimGateway_1 pending claims =   142
AssessorThread_1  >>  approve  74 claims from ClaimGateway_2     ClaimGateway_2 pending claims =   147
AssessorThread_0  >>  approve  15 claims from ClaimGateway_2     ClaimGateway_2 pending claims =   132
AssessorThread_0  >>  pending payments =  15
AssessorThread_2  >>  pending payments =  36
AssessorThread_1  >>  pending payments =  74
AssessorThread_1  >>  issue  74 payments to PaymentGateway_1     PaymentGateway_1 tokens =   6
AssessorThread_0  >>  issue   6 payments to PaymentGateway_1     PaymentGateway_1 tokens =   0
AssessorThread_2  >>  issue   0 payments to PaymentGateway_1     PaymentGateway_1 tokens =   0
            main  >>
```

AssessorThread_0: unsuccessful payments (15-6 = 9)
are added to pending payments of next days

```
            main  >>  ================================================================================
            main  >>  Day 5
            main  >>  ClaimGateway_0 pending claims =   219
            main  >>  ClaimGateway_1 pending claims =   142
            main  >>  ClaimGateway_2 pending claims =   132
            main  >>  reset PaymentGateway_0 tokens =    80
            main  >>  reset PaymentGateway_1 tokens =    80
            main  >>
  BrokerThread_0  >>  new claims =  24     split to [24]
  BrokerThread_2  >>  new claims =  56     split to [50, 6]
  BrokerThread_1  >>  new claims =  83     split to [50, 33]
  BrokerThread_1  >>  submit  50 claims to ClaimGateway_1       ClaimGateway_1 pending claims =   192
  BrokerThread_1  >>  submit  33 claims to ClaimGateway_1       ClaimGateway_1 pending claims =   225
  BrokerThread_2  >>  submit  50 claims to ClaimGateway_0       ClaimGateway_0 pending claims =   269
  BrokerThread_2  >>  submit   6 claims to ClaimGateway_0       ClaimGateway_0 pending claims =   275
  BrokerThread_0  >>  submit  24 claims to ClaimGateway_2       ClaimGateway_2 pending claims =   156
  BrokerThread_0  >>
AssessorThread_2  >>  approve  75 claims from ClaimGateway_2    ClaimGateway_2 pending claims =    81
AssessorThread_1  >>  approve  66 claims from ClaimGateway_0    ClaimGateway_0 pending claims =   209
AssessorThread_0  >>  approve   7 claims from ClaimGateway_0    ClaimGateway_0 pending claims =   202
AssessorThread_0  >>  pending payments =  16
AssessorThread_2  >>  pending payments = 111
AssessorThread_1  >>  pending payments =  66
AssessorThread_1  >>  issue  66 payments to PaymentGateway_1    PaymentGateway_1 tokens =  14
AssessorThread_0  >>  issue  16 payments to PaymentGateway_0    PaymentGateway_0 tokens =  64
AssessorThread_2  >>  issue  14 payments to PaymentGateway_1    PaymentGateway_1 tokens =   0
            main  >>
            main  >>  ================================================================================
            main  >>  Day 6
            main  >>  ClaimGateway_0 pending claims =   202
            main  >>  ClaimGateway_1 pending claims =   225
            main  >>  ClaimGateway_2 pending claims =    81
            main  >>  reset PaymentGateway_0 tokens =    80
            main  >>  reset PaymentGateway_1 tokens =    80
            main  >>
  BrokerThread_2  >>  new claims =  70     split to [50, 20]
  BrokerThread_0  >>  new claims =  65     split to [50, 15]
  BrokerThread_1  >>  new claims =  99     split to [50, 49]
  BrokerThread_1  >>  submit  50 claims to ClaimGateway_1       ClaimGateway_1 pending claims =   275
  BrokerThread_1  >>  submit  49 claims to ClaimGateway_1       ClaimGateway_1 pending claims =   324
  BrokerThread_2  >>  submit  50 claims to ClaimGateway_2       ClaimGateway_2 pending claims =   131
  BrokerThread_2  >>  submit  20 claims to ClaimGateway_2       ClaimGateway_2 pending claims =   151
  BrokerThread_0  >>  submit  50 claims to ClaimGateway_1       ClaimGateway_1 pending claims =   374
  BrokerThread_0  >>  submit  15 claims to ClaimGateway_1       ClaimGateway_1 pending claims =   389
  BrokerThread_0  >>
AssessorThread_2  >>  approve  62 claims from ClaimGateway_0    ClaimGateway_0 pending claims =   140
AssessorThread_1  >>  approve  57 claims from ClaimGateway_0    ClaimGateway_0 pending claims =    83
AssessorThread_0  >>  approve  33 claims from ClaimGateway_1    ClaimGateway_1 pending claims =   356
AssessorThread_0  >>  pending payments =  33
AssessorThread_1  >>  pending payments =  57
AssessorThread_2  >>  pending payments = 159
AssessorThread_2  >>  issue  80 payments to PaymentGateway_0    PaymentGateway_0 tokens =   0
AssessorThread_1  >>  issue  57 payments to PaymentGateway_1    PaymentGateway_1 tokens =  23
AssessorThread_0  >>  issue   0 payments to PaymentGateway_0    PaymentGateway_0 tokens =   0
            main  >>
            main  >>  ================================================================================
            main  >>  Summary
            main  >>  AssessorThread_2 successful payments =   213
            main  >>  AssessorThread_1 successful payments =   212
            main  >>  AssessorThread_0 successful payments =    36
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

AssessorThread_0: pending payments
are accumulated from previous days